College of Computer Science and Information Technology
Department of Computer Engineering

ARTI403: Image Processing

Lab 2: Digital Image Fundamentals

Student Name: Saad Abdulaziz Alshehri
Student ID: 2240006495

Import the libraries and load the images

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img1 = cv2.imread('images/camera.png', cv2.IMREAD_GRAYSCALE)
img2 = cv2.imread('images/coins.png', cv2.IMREAD_GRAYSCALE)

# the two images must have the same size for the operations
img2 = cv2.resize(img2, (img1.shape[1], img1.shape[0]))
print(img1.shape, img2.shape)

fig, axes = plt.subplots(1, 2, figsize=(10, 5))
axes[0].imshow(img1, cmap='gray', vmin=0, vmax=255)
axes[0].set_title('Image 1 (camera)')
axes[1].imshow(img2, cmap='gray', vmin=0, vmax=255)
axes[1].set_title('Image 2 (coins)')
for ax in axes:
    ax.axis('off')
plt.show()
```

```
(512, 512) (512, 512)
```

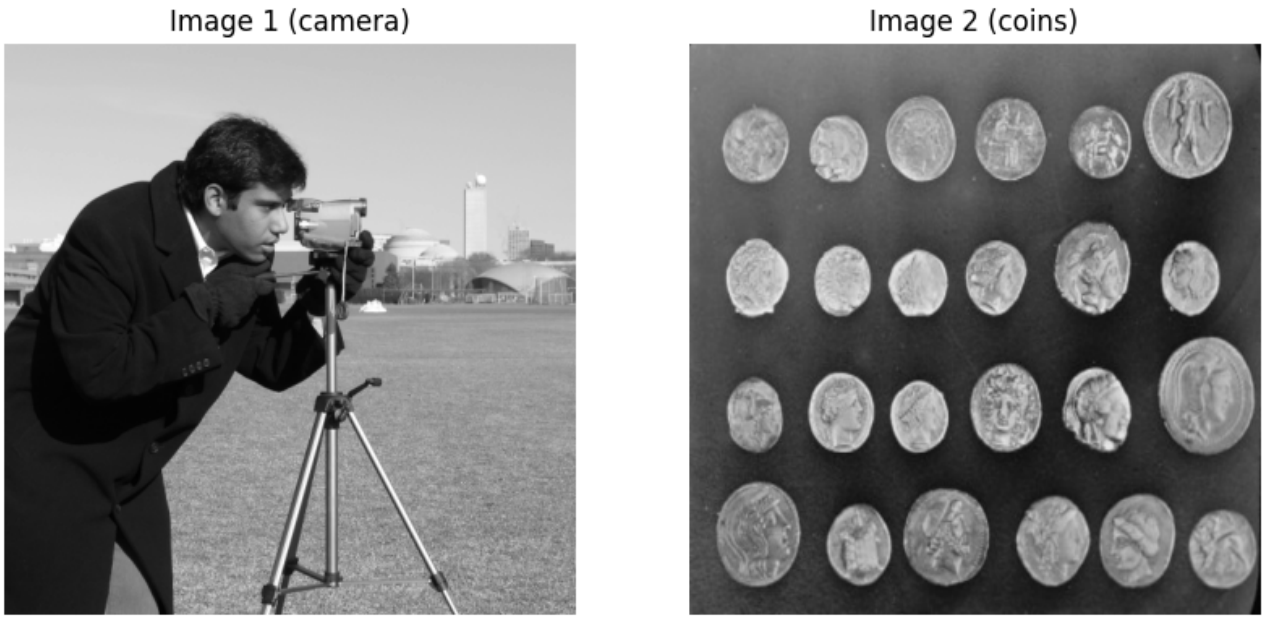


Image Sampling and Quantization

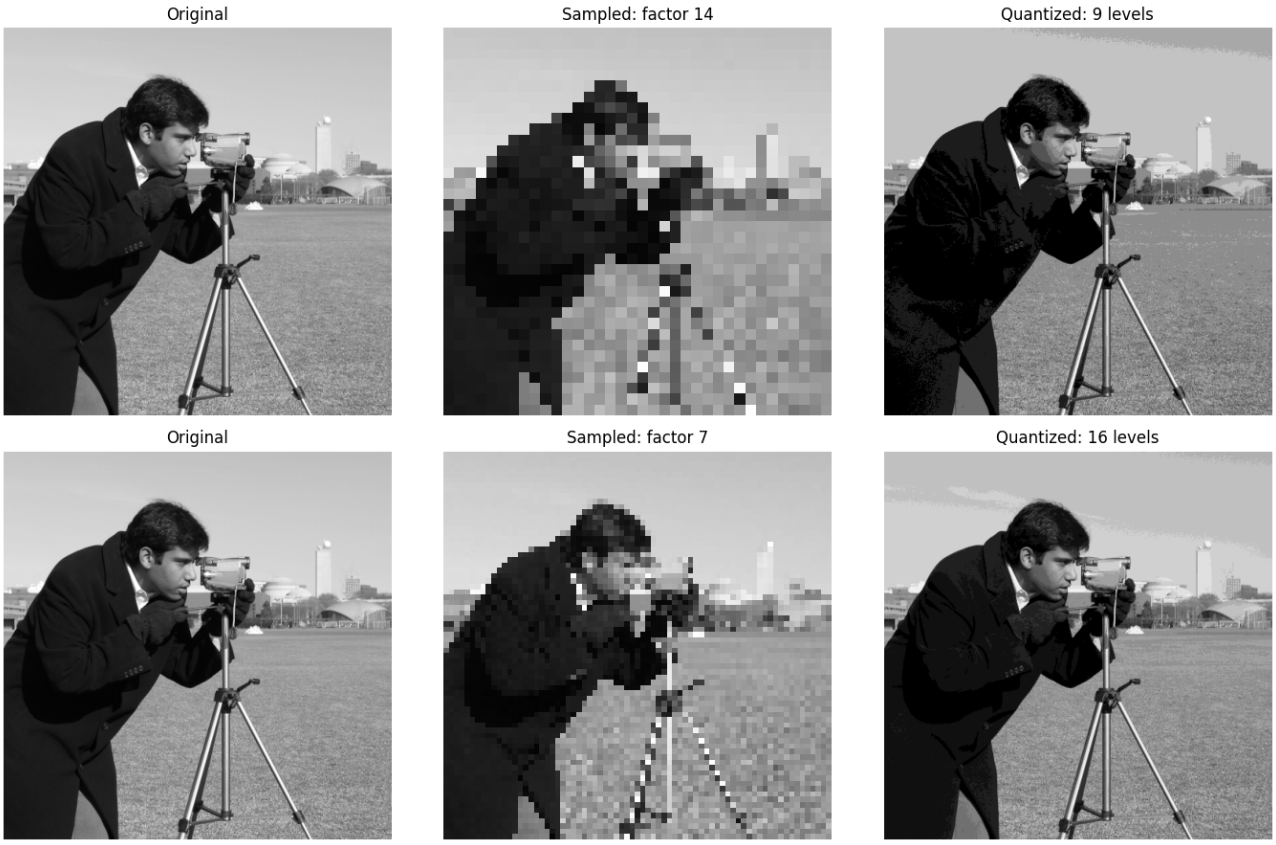
Task 1: change the sampling and quantization parameters

```
def sample_image(image, factor):
    height, width = image.shape[:2]
    return cv2.resize(image, (width // factor, height // factor), interpolation=cv2.INTER_NEAREST)

def quantize_image(image, levels):
    step = 256 // levels
    quantized = np.floor(image / step) * step
    return quantized.astype(np.uint8)
```

```
# first row: values from the manual, second row: my values
sampled_1 = sample_image(img1, 14)
quantized_1 = quantize_image(img1, 9)
sampled_2 = sample_image(img1, 7)
quantized_2 = quantize_image(img1, 16)

fig, axes = plt.subplots(2, 3, figsize=(14, 9))
axes[0, 0].imshow(img1, cmap='gray', vmin=0, vmax=255)
axes[0, 0].set_title('Original')
axes[0, 1].imshow(sampled_1, cmap='gray', vmin=0, vmax=255)
axes[0, 1].set_title('Sampled: factor 14')
axes[0, 2].imshow(quantized_1, cmap='gray', vmin=0, vmax=255)
axes[0, 2].set_title('Quantized: 9 levels')
axes[1, 0].imshow(img1, cmap='gray', vmin=0, vmax=255)
axes[1, 0].set_title('Original')
axes[1, 1].imshow(sampled_2, cmap='gray', vmin=0, vmax=255)
axes[1, 1].set_title('Sampled: factor 7')
axes[1, 2].imshow(quantized_2, cmap='gray', vmin=0, vmax=255)
axes[1, 2].set_title('Quantized: 16 levels')
for ax in axes.flat:
    ax.axis('off')
plt.tight_layout()
plt.show()
```



A bigger sampling factor gives a smaller image with less detail. More quantization levels keep more gray shades.

Arithmetic Operations

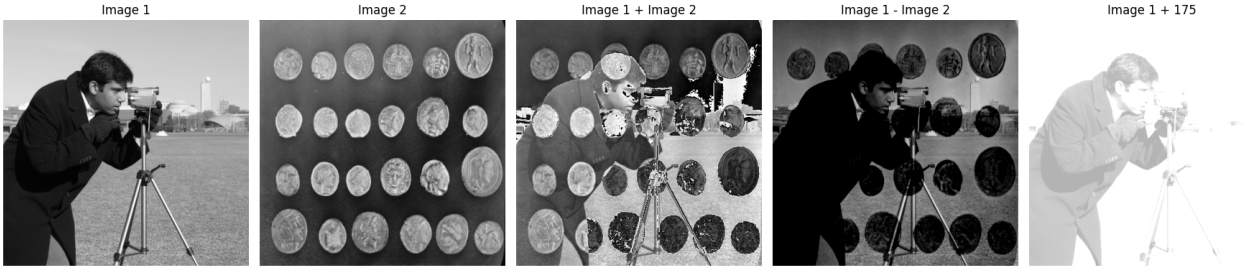
Task 2: subtract two images and add a constant (175)

```
im1arr = np.asarray(img1)
im2arr = np.asarray(img2)

addition = im1arr + im2arr                          # example from the manual
subtraction = cv2.subtract(im1arr, im2arr)          # subtract the two images
add_175 = cv2.add(im1arr, np.full_like(im1arr, 175)) # add the constant 175

fig, axes = plt.subplots(1, 5, figsize=(18, 4))
for ax, im, t in zip(axes, [im1arr, im2arr, addition, subtraction, add_175],
                     ['Image 1', 'Image 2', 'Image 1 + Image 2', 'Image 1 - Image 2', 'Image 1 + 175']):
```

```
    ax.imshow(im, cmap='gray', vmin=0, vmax=255)
    ax.set_title(t)
    ax.axis('off')
plt.tight_layout()
plt.show()
```



Sets and Logical Operations

Task 2: set difference, symmetric difference and intersection

```
union = np.bitwise_or(im1arr, im2arr)                                  # example from the manual
set_difference = np.bitwise_and(im1arr, np.bitwise_not(im2arr))   # A - B
symmetric_difference = np.bitwise_xor(im1arr, im2arr)
intersection = np.bitwise_and(im1arr, im2arr)

fig, axes = plt.subplots(2, 3, figsize=(14, 9))
for ax, im, t in zip(axes.flat, [im1arr, im2arr, union, set_difference, symmetric_difference, intersection],
                     ['A (Image 1)', 'B (Image 2)', 'Union', 'Set difference', 'Symmetric difference', 'Intersection']):
    ax.imshow(im, cmap='gray', vmin=0, vmax=255)
    ax.set_title(t)
    ax.axis('off')
plt.tight_layout()
plt.show()
```

